COMPUTER ORGANIZATION AND ARCHITECTURE

MODULE 1 - PART 1

COURSE CODE : PBCST404

SEMESTER – S4 (GROUP A)

2024 SCHEME

SYLLABUS

MODULE 1

BASIC STRUCTURE OF COMPUTERS :- Functional units - Basic operational concepts; Memory map; Endianness. **CISC VS RISC ARCHITECTURES**:- RISC Introduction - Assembly Language, Assembler directives, Assembling. **PROGRAMMING CONCEPTS** - Program flow, Branching, Conditional statements, Loops, Arrays, Function calls; Instruction execution cycle.
MACHINE LANGUAGE - Instructions, addressing modes, Stored program concept. Evolution of the RISC Architecture.

MODULE 1 :

Basic Structure of computers :– Functional units - Basic operationalconcepts; Memory map; Endianness.

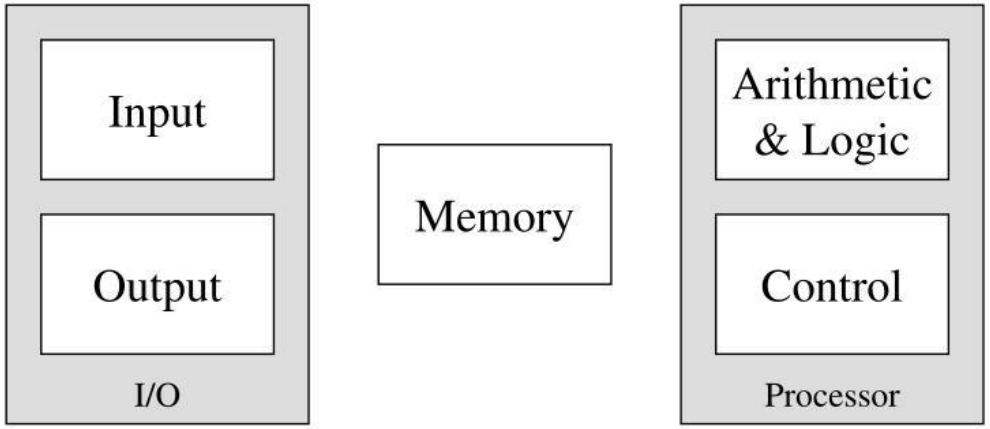
CISC vs RISC architectures:- RISC Introduction - Assembly Language, Assembler directives, Assembling.

Programming concepts :- Program flow, Branching, Conditional statements, Loops, Arrays, Function calls; Instruction execution cycle.

Machine language :- Instructions, addressing modes, Stored program concept. Evolution of the RISC Architecture.

FUNCTIONAL UNITS

Computer Functional Units



Input Unit

- The method of feeding **data and programs** into a computer is performed by **input devices**.
- An input device reads data from a source (for example, magnetic disks, keyboard entry, etc.) and **translates it into electronic signals using ADC (Analog-to-Digital Conversion)**. These signals are transferred into the **CPU for processing**.

Examples of Input Devices

- Keyboard
- Mouse
- Scanner
- Magnetic disk drives
- Barcode readers
- Touch screens

Output Unit

- Computer **output devices** convert the processed digital signals into a **human-readable form** using **DAC (Digital-to-Analog Conversion)**.
- The output unit sends processed results from the computer to the external environment (users).

Examples of Output Devices

- Display screens / Monitors
- Printers
- Plotters
- Microfilm devices
- Speakers / Synthesizers
- High-tech interactive boards (Smart Boards)
- Film recorders

Memory Unit (MU)

- A **Memory Unit** is a collection of storage cells along with associated circuits required to store and retrieve information.
- Memory provides a storage medium for data and programs used by the computer.
- It performs one of the most fundamental functions of a computer **data storage and fast access to information**.
- Memory storage exists in different levels depending on speed and purpose.
- The most common form is **Primary Storage**, typically referring to **RAM (Random Access Memory)** the main working memory of the computer that holds data and applications currently being used.

- There is **Secondary Storage**, which refers to **external storage devices and media** used for long-term data retention.
- Secondary memory is non-volatile, meaning data is not lost when power is turned off.
- It is used to store large amounts of information that must be preserved permanently.

Examples of Secondary Storage:

- Hard Drives (HDD)
- Solid State Drives (SSD)
- Optical Media (CD / DVD / Blu-ray)
- USB Flash Drives / Pen Drives
- Memory Cards
- Magnetic Tapes
- Cloud Storage

These devices store data that is **not currently being processed** by the CPU but may be required later for future use.

Arithmetic Logic Unit (ALU)

- When data is entered through an input device, it is first stored in the **Primary Storage Unit (main memory)**.
- The **Arithmetic Logic Unit (ALU)** is responsible for performing all the actual processing of data and instructions.
- It executes:

  Arithmetic operations – addition, subtraction, multiplication, and division

  Logical operations – comparison and decision-making operations

Whenever processing is required, data is transferred from the memory to the ALU. After completion, the processed output is sent back to the storage unit either for further processing or for storing the result.

Control Unit (CU)

The **Control Unit (CU)** functions as the controller or supervisor of computer operations. It **coordinates and manages** the activities of all components of the computer system.

Its key functions are:

- Determining the sequence in which instructions and programs are executed
- Interpreting instructions stored in the main memory
- Issuing control signals to other components to execute instructions
- Managing input and output operations by coordinating peripheral devices

The Control Unit acts like a **switchboard operator**, especially when multiple users or devices interact with the system at the same time.

Central Processing Unit (CPU)

- The **CPU** is made up of three main components:

  Arithmetic Logic Unit (ALU)

  Control Unit (CU)

  Memory Unit (MU)

- ✓ Together, they form the **Central Processing Unit**, often called the **brain of the computer system**. The CPU performs all the major calculations
- ✓ Makes decisions
- ✓ Controls and directs the operations of all other parts of the computer
- The CPU is therefore the **central component responsible for processing and control** in a computer.

BASIC OPERATIONAL CONCEPTS

To perform a task, a program containing a **sequence of instructions** is stored in memory. Each instruction is fetched by the processor, interpreted, and then executed.

Instruction Execution Example

- **Load LOC, R2**

  Fetches the operand from memory location **LOC** and loads it into **register R2**.

- **Add R1, R2, R3**

  Adds the contents of registers **R1** and **R2**, and stores the result in **R3**.

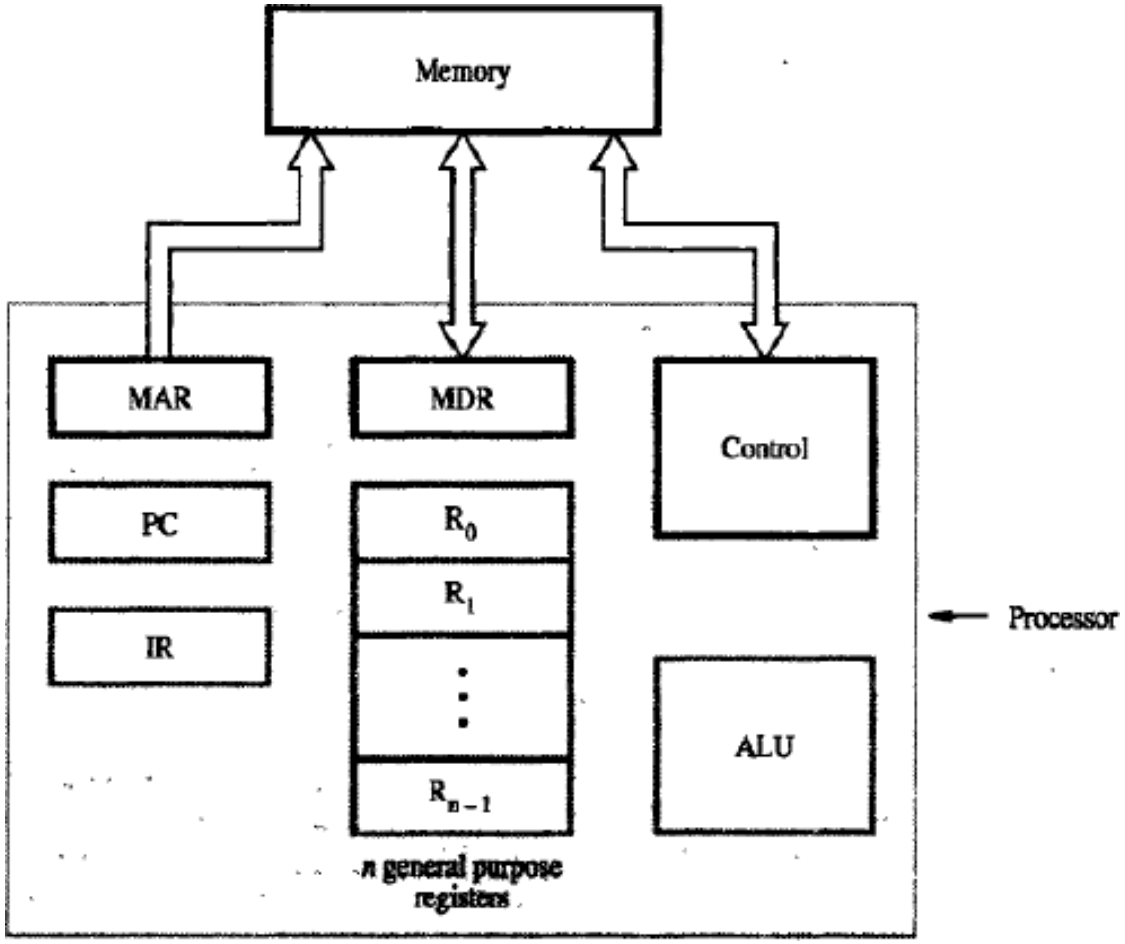
- **Store R4, LOC**

  Copies the value from **register R4** into the memory location **LOC**.

- **Add R1, R0**

  Adds the contents of **R1** and **R0**, and stores the result back in **R0**.

PROCESSOR AND MEMORY CONNECTION



The processor consists of:

- **ALU (Arithmetic Logic Unit)**
- **Control Unit**
- **Registers**

Important registers include:

- **Instruction Register (IR)** – Holds the instruction currently being executed.

  The instruction is **decoded and executed** from IR

  Used **internally by the CPU** during execution

- **Program Counter (PC)** – Contains the address of the next instruction to be fetched.

  Automatically **updates after each instruction**

  Modified during **branch and jump** instructions

- **General-Purpose Registers (R0 to Rn−1)** – Used to store temporary data, operands, and intermediate results.

Operating Steps of a Program

1. Program is loaded into memory using input devices.
2. Program Counter (PC) is initialized with the address of the first instruction.
3. The address in PC is transferred to Memory Address Register (MAR) and a read signal is given.
4. The instruction is fetched from memory into Memory Data Register (MDR).
5. The instruction is transferred from MDR to Instruction Register (IR).
6. The instruction is decoded and executed by the CPU.
   - If operands are needed, they are fetched from memory (MAR → MDR → ALU).
7. The ALU performs the required operation and stores the result in a register.
8. If required, the result is written back to memory (MAR, MDR, write signal).
9. The Program Counter (PC) is incremented to point to the next instruction.

 “This process is called the fetch-decode-execute cycle.”

INPUT/OUTPUT OPERATIONS

In addition to memory transfers:

- The processor **receives data from input devices**.
- It **sends processed results to output devices**.

INTERRUPTS

Interrupts are signals that make the processor respond immediately to an event.
An external device can send an interrupt signal to request attention from the CPU.

Working of Interrupts

1. The current program execution is temporarily suspended.
2. The processor jumps to a predefined program called **Interrupt Service Routine (ISR)**.
3. The ISR handles the required task or event.
4. After completing the task, control returns to the main program.

BUS STRUCTURES

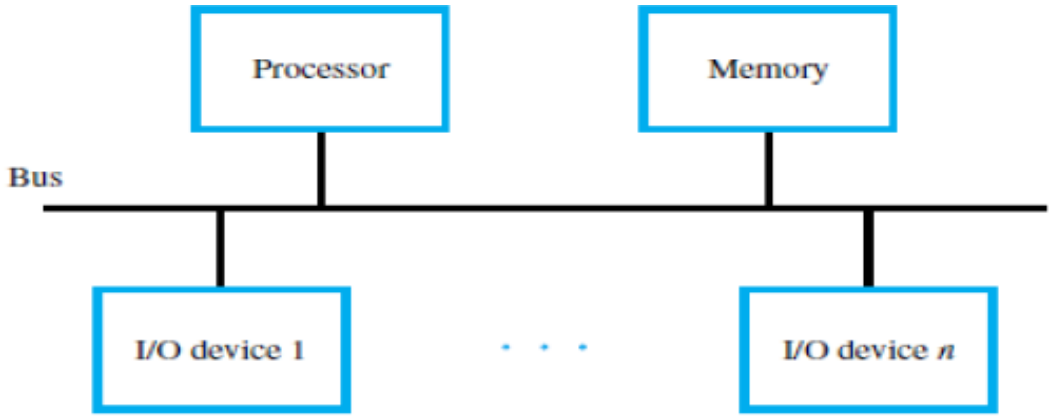
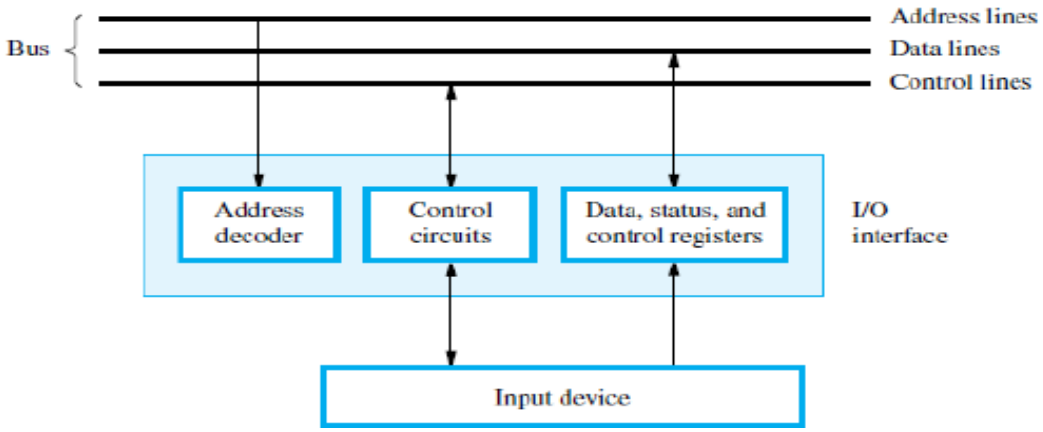


Figure 2



BUS STRUCTURES

A bus is an interconnection network used to connect different parts of a computer system.
It consists of **address lines, data lines, and control lines**.

Features of Bus Structure

- Only one source and destination can use the bus at a time.
- I/O devices are connected to the bus for communication.
- The processor sends control signals through control lines.
- Data is transferred using data lines.
- Address lines carry the address of the device.
- All devices check the address using address decoders.
- The device with matching address responds to the processor.

“Bus structure enables communication between CPU, memory, and I/O devices.”

MEMORY-MAPPED I/O

Memory-mapped I/O is a method where I/O devices and memory share the same address space.

- I/O devices are assigned unique addresses just like memory.
- Same instructions used for memory can be used for I/O devices.
- Processor uses control lines to perform Read and Write operations.
- Data transfer happens through data lines.
- Each device has interface registers with a unique address.
- The device interface includes:
    - Address decoder
    - Data and status registers
    - Control circuitry

“Memory-mapped I/O allows I/O devices to be accessed like memory locations.”

MEMORY LOCATIONS AND ADDRESSES

Memory consists of a large number of storage cells, each storing a single bit (0 or 1).

- Memory is organized into groups of bits called **words**.
- A word contains **n bits**, and this is called **word length** (usually 16–64 bits).
- Memory can be viewed as a collection of words, each having a **unique address**.
- A group of 8 bits is called a **byte**.
- Instructions can access data in the form of **words or bytes**.
- Each memory location must have a **unique address**.
- If a memory has **2^k locations**, it will have **2^k unique addresses** (address space).

"Memory is organized as addressable locations where each location stores data and is identified by a unique address."

Byte Addressability

Byte addressability means each byte in memory has a unique address.

- Modern computers use **byte-addressable memory**.
- Each byte is assigned a **unique address** (0, 1, 2, …).
- A word consists of multiple bytes (depending on word length).
- Example:
  - In a **32-bit system (4 bytes)**
  - Word addresses → 0, 4, 8, 12 …
- Words are stored as a **sequence of bytes**.
- This method avoids giving addresses to each bit (only bytes are addressed).

"Byte addressability allows each byte in memory to be accessed using a unique address."

Byte Addressable Memory means:

- Each memory address stores 1 byte (8 bits).
- Every byte has a unique address.
- Larger data (word, int) uses multiple consecutive addresses.
-

Simple definition:

In byte addressable memory, each memory location stores one byte, and every byte has its own address.

Example:

- A 32-bit integer (4 bytes) needs 4 memory addresses.

Endianness

Q > Explain the two different types of byte assignment (Big Endian and Little Endian) using neat diagrams.

Byte Assignment (Endianness):

Endianness is the rule that decides **which byte is stored first in memory** when data uses more than one byte.

Example: an **integer** like 1234 needs **4 bytes**.

There are **two types**:

1. **Big Endian**
2. **Little Endian**

1) Big Endian

Meaning: Most Significant Byte (MSB) stored at the lowest address.

Assume 32-bit value: **0x12 34 56 78**
Starting address: **0x1000**

Address		**Data**	
0x1000	→	12	(MSB)
0x1001	→	34	
0x1002	→	56	
0x1003	→	78	(LSB)

2) Little Endian

Meaning: Least Significant Byte (LSB) stored at the lowest address.

Same value: **0x12 34 56 78**
Starting address: **0x1000**

Address		**Data**	
0x1000	→	78	(LSB)
0x1001	→	56	
0x1002	→	34	
0x1003	→	12	(MSB)

Feature	Big Endian	LittleEndian
Lowest address	MSB	LSB
Human readability	Easy	Hard
RISC-V uses	✕	✓

Big Endian

Advantages

- Easy to read and understand (MSB comes first)
- Matches normal human number representation

✕Disadvantages

- Less efficient for some CPU operations
- Not commonly used in modern systems

Little Endian

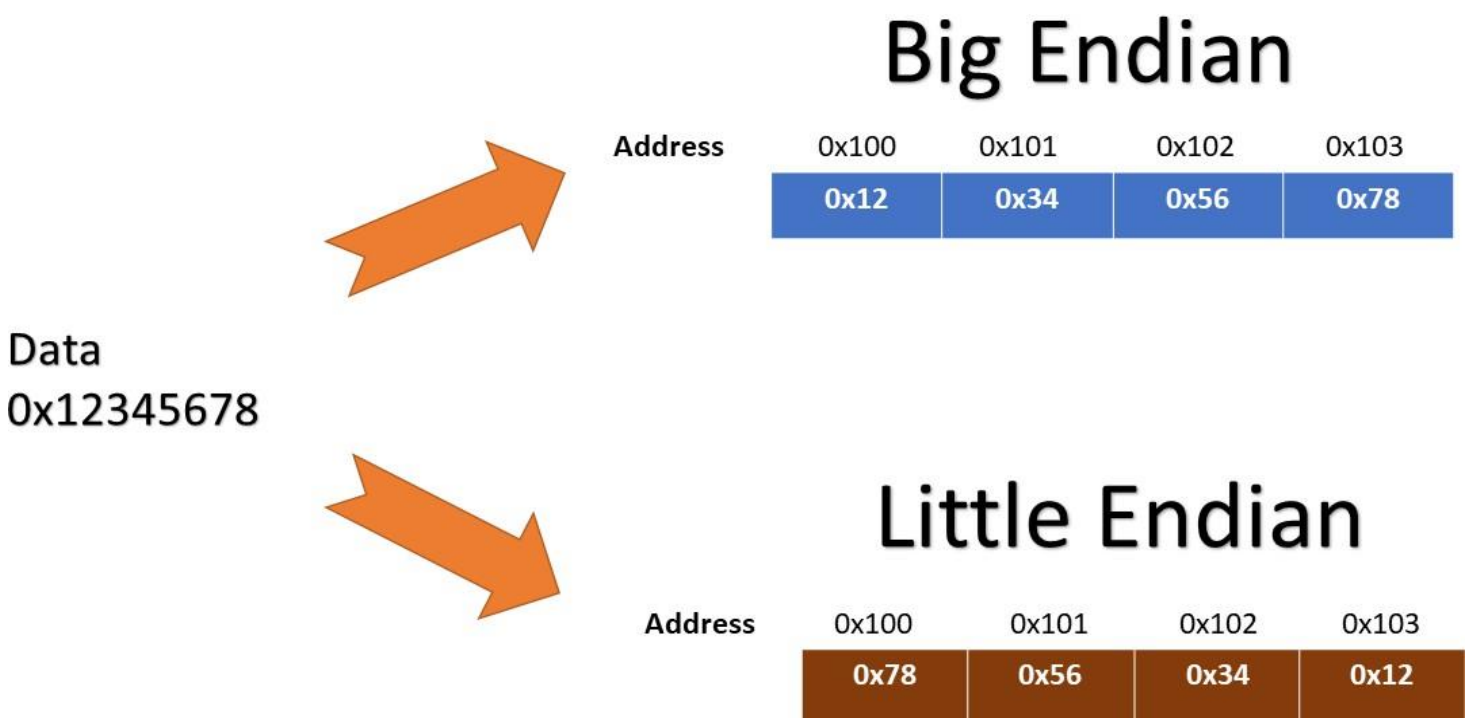
Advantages

- Faster processing for CPUs
- Easy to handle lower bytes first

✕Disadvantages

- Harder for humans to read
- Confusing during data transfer between systems

ENDIANESS : BIG ENDIAN AND LITTLE ENDIAN



Memory Map

A memory map is the organization of computer memory into different sections.
It shows how memory is divided for instructions, data, stack, heap, operating system, and I/O devices.

Memory is divided into different segments for proper organization
• Each segment has a specific function

Segments of Memory Map

1. **Text Segment**
- Stores program instructions (machine code)
- May include constants and read-only data

2. **Global Data Segment**
- Stores global variables (a variable that can be used everywhere in the program)
- Accessible by all functions
- Accessed using global pointer register (gp)

3. **Dynamic Data Segment**

- Contains stack and heap
- Memory is allocated during execution

Stack:

- Stores local variables and function calls
- Works in LIFO
- Grows downward

Heap:

- Stores dynamically allocated memory
- Grows upward

4. **Exception Handler Segment**

- Stores interrupt/exception handling code
- Executes during errors or interrupts

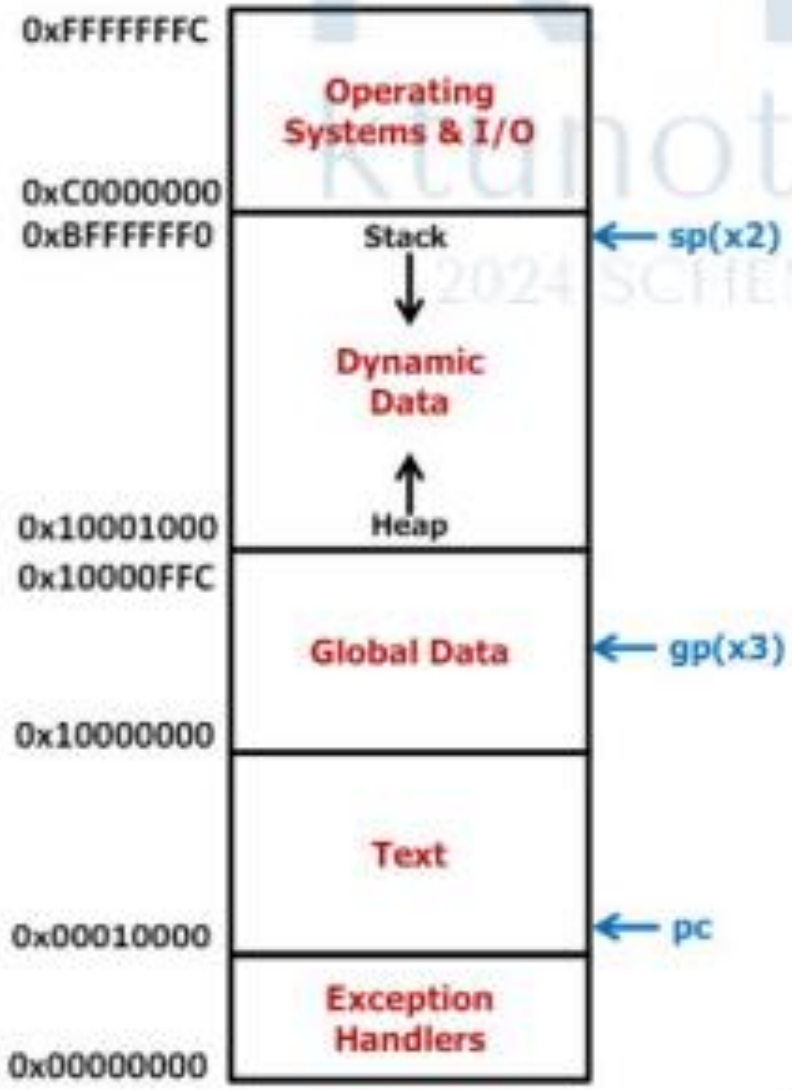
5. **Operating System and I/O Segment**

- Reserved for OS and I/O devices
- Not accessible by user programs

⍰ Functions of Memory Map

- Defines where programs are loaded
- Shows location of data and devices
- Helps CPU access memory correctly
- Used by OS for memory management

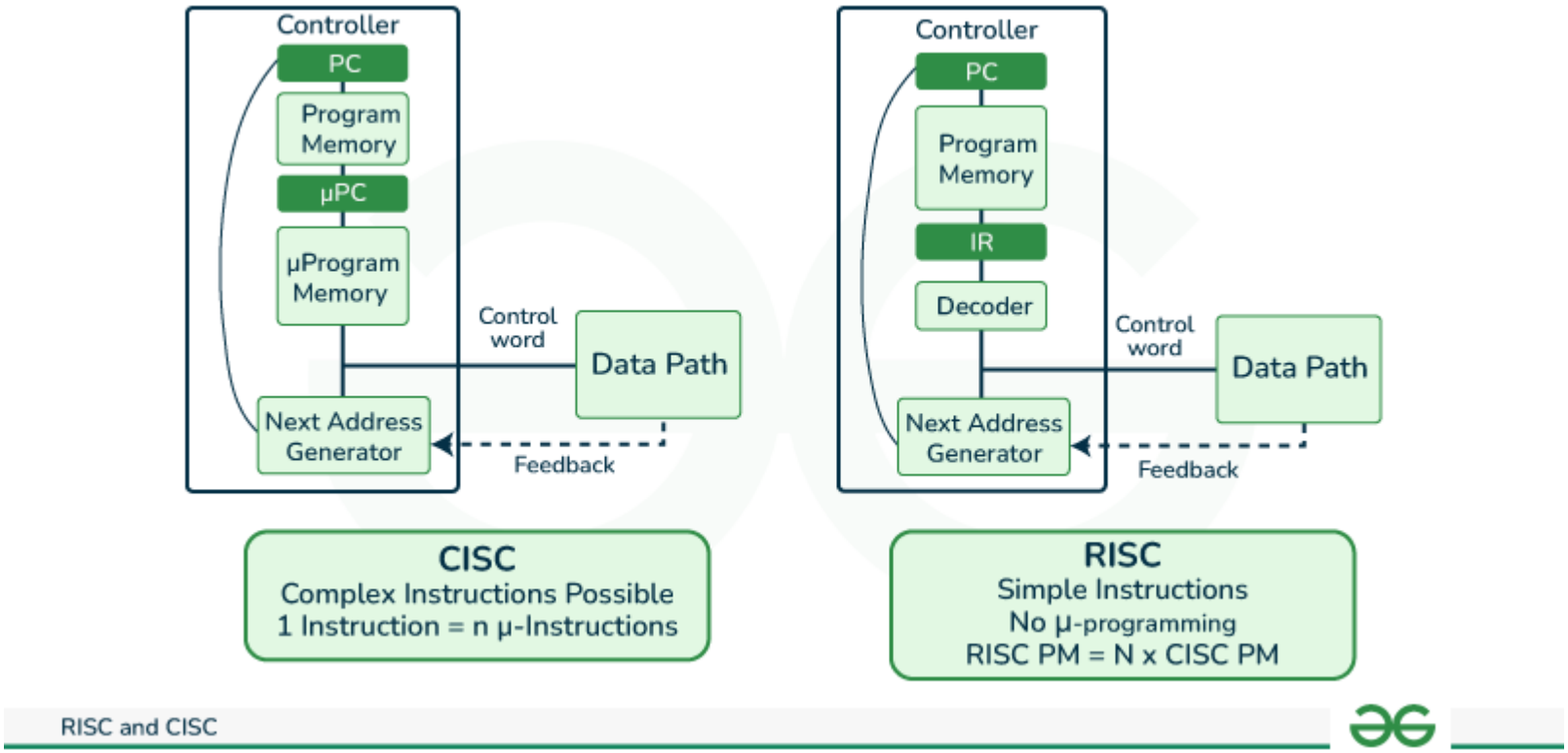
“Memory map organizes memory into segments for efficient execution and management.”


0xFFFFFFFC
Operating Systems & I/O
0xC0000000
0xBFFFFFF0
Stack
sp(x2)
Dynamic Data
0x10001000
Heap
0x10000FFC
Global Data
gp(x3)
0x10000000
Text
0x00010000
pc
Exception Handlers
0x00000000

RISC vs CISC:

RISC (Reduced Instruction Set Computer) and CISC (Complex Instruction Set Computer) architectures.

Feature	**RISC**	**CISC**
Instruction set	Small, simple	Large, complex
Instruction length	Fixed (usually 32-bit)	Variable
Execution	One instruction per cycle (mostly)	Multiple cycles per instruction
Memory access	Load/Store only	Instructions can access memory
Registers	Large register file	Fewer registers
Hardware design	Simple	Complex
Examples	**RISC-V, ARM**	**x86, Intel 8086**



RISC vs CISC

- Two different approaches to designing processors
- **RISC** uses a **small set of simple, fixed-length instructions** that usually execute in **one clock cycle**
- **CISC** uses a **large set of complex instructions** that can perform **multiple operations in one instruction**
- **RISC → faster execution** due to simple instructions
- **CISC → takes multiple clock cycles** to execute complex instructions

RISC (Reduced Instruction Set Architecture)

- RISC simplifies processor design by using a **small and uniform set of instructions**
- Each instruction performs a **basic operation** (load, compute, store)
- Instructions are designed to execute in **one clock cycle**
- This makes **pipelining easy and efficient**

⍰ Characteristics of RISC

- Simple instructions → **easy decoding**
- Fixed-length instructions (one word)
- Executes in **single clock cycle**
- Uses **more general-purpose registers**
- Uses **simple addressing modes**
- Supports **efficient pipelining**

RISC-V Registers are **small, fast storage locations inside the CPU** used to hold data, addresses, an instructions during program execution.

RISC-V registers are CPU memory units that temporarily store operands, results, and addresses to perform operations efficiently.

Key points:

RISC-V has **32 general-purpose registers**
Named **x0 to x31**
All registers are **same size**
32-bit in RV32
64-bit in RV64

General-Purpose Registers in RISC-V (32 Registers)

RISC-V provides **32 General-Purpose Registers (GPRs)** used to store data, addresses, and intermediate results.

Key points

Registers are named **x0 to x31**
All are **same size**
32-bit in RV32
64-bit in RV64
Used for **arithmetic, logic, data movement, and addressing**
x0 is hard-wired to 0

x Register	ABI Name	Purpose
x0	zero	Constant 0
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Thread pointer

x Register	ABI Name	Purpose
x5–x7	t0–t2	Temporary
x8	s0 / fp	Saved / Frame pointer
x9	s1	Saved
x10–x17	a0–a7	Arguments / Return values
x18–x27	s2–s11	Saved registers
x28–x31	t3–t6	Temporary

CISC (Complex Instruction Set Architecture)

 Idea:

CISC uses big and powerful instructions to do more work in one step.

 One instruction can do:

load + calculate + store

 Characteristics (simple & cute)

- Uses complex instructions → decoding is difficult
  - Instructions are long / variable size
  - Takes multiple clock cycles
  - Uses fewer registers (works directly with memory)
  - Has many complex addressing modes

“CISC uses complex, multi-step instructions to reduce program size but increases execution time.”

CPU Performance (RISC vs CISC)

□ Both **RISC and CISC** try to **reduce CPU time** □

□ CPU Time Formula

□ **CPU Time = Instruction Count × CPI × Clock Cycle Time**

□ Terms (easy meaning)

1 Instruction Count

□ Total number of instructions in a program
✔ Depends on program & instruction set

2 CPI (Cycles Per Instruction)

□ Number of cycles needed for one instruction
✔ RISC → low CPI □
✔ CISC → high CPI

3 Clock Cycle Time

 Time for one clock cycle
✔ Smaller = faster CPU

 “CPU performance depends on Instruction Count, CPI, and Clock Cycle Time, and RISC & CISC optimize these differently.”

- **RISC (Reduced Instruction Set Computer):**

  Focuses on reducing the number of **cycles per instruction**, even if this results in a higher number of instructions per program. Simpler instructions enable faster execution and better pipelining.

- **CISC (Complex Instruction Set Computer):**

  Aims to reduce the **number of instructions per program** by using more complex instructions, often at the expense of an increased **number of cycles per instruction**.

Example:

Suppose we have to add two 8-bit numbers:

- **CISC approach:** There will be a single command or instruction for this like ADD which will perform the task.

- **RISC approach:** Here programmer will write the first load command to load data in registers then it will use a suitable operator and then it will store the result in the desired location.

- So, add operation is divided into parts i.e. load, operate, store due to which RISC programs are longer and require more memory to get stored but require fewer transistors due to less complex command.

□ CISC (single step)

```
ADD A, B
```

□ One instruction does everything □

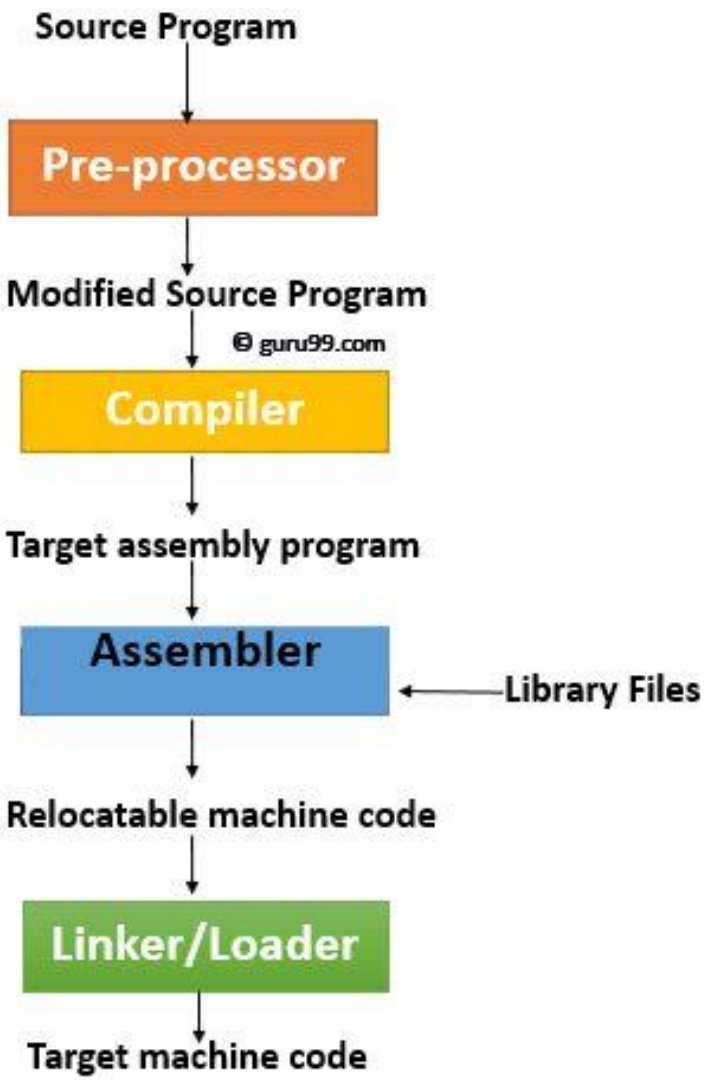
□ RISC (multiple steps)

```
LOAD R1, A
LOAD R2, B
ADD  R3, R1, R2
STORE R3, C   //"Take the value inside register R3 and save it into memory location C."
```

□ Load → Add → Store

RISC	CISC
Small and simple instruction set	Large and complex instruction set
Fixed-length instructions	Variable-length instructions
Usually 1 cycle per instruction	Multiple cycles per instruction
More general-purpose registers	Fewer registers
Simple and limited addressing modes	Complex and many addressing modes
Larger code size (more instructions)	Smaller code size (fewer instructions)
Simple hardware, easier pipelining	Complex hardware, harder pipelining
Examples: ARM, RISC-V, MIPS	Examples: x86, Intel 80386

Compilers, Assemblers, Linkers and Loaders – what are they?



Program Translation & Execution Flow

User writes a program in **C / C++** (high-level language)

Compiler

- Translates high-level code into **assembly language**
- Checks syntax and reports errors

Assembler

- Converts assembly code into **machine code**
- Generates **object files**

Linker

- Combines multiple object files
- Links required **libraries**
- Produces a single **executable file**

Loader

- Loads program into **main memory**
- Allocates memory and resolves addresses

Execution

- CPU starts executing the program instructions

Assembling

Assembling means converting **assembly language → machine code (0 & 1)**

⍰ Steps in Assembling

⍰ 1. Source Code

- Programmer writes code in assembly language
- Uses simple commands like ADD, MOV

⍰ 2. Assembler ☆

- Reads the program
- Converts each instruction into machine code
- Also handles memory using directives

⍰ 3. Object File

- Output is an **object file (.o)**
- Contains machine code + address details

⍰ 4. Linking

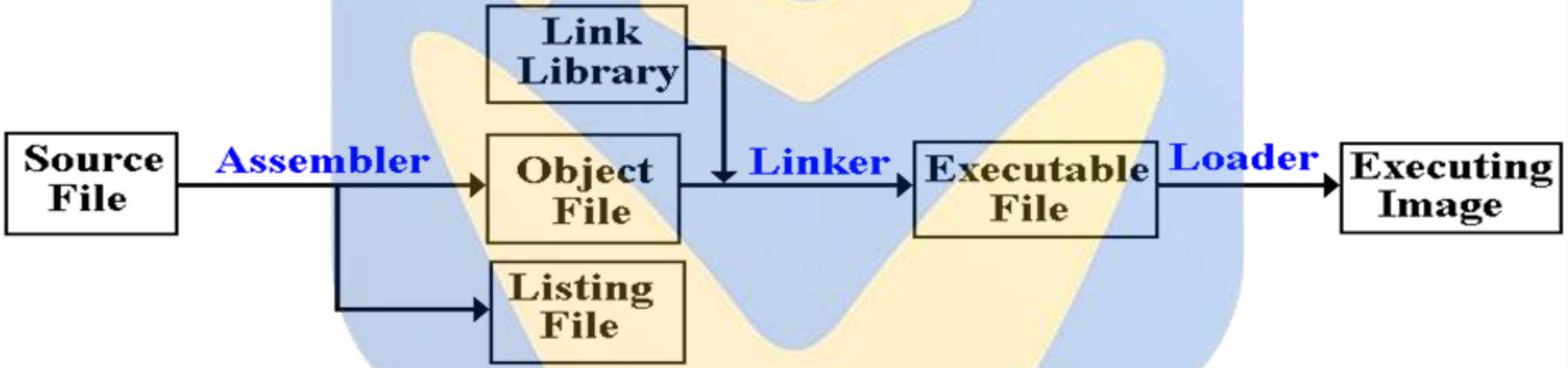
- Joins object files together
- Adds library functions

- Creates final executable file

 5. Execution

- Program is loaded into memory
- CPU executes the instructions

“Assembling converts assembly code into machine code and prepares it to run.”


Source File
Assembler
Object File
Listing File
Link Library
Linker
Executable File
Loader
Executing Image

KOS

Assembly Language

- Assembly language is a human-readable form of machine language
- Uses mnemonics like ADD, SUB
- Each instruction has:
  - ✔ Operation
  - ✔ Operands

- Instructions
- Instructions perform operations like addition, subtraction

Example:

- add a, b, c → a = b + c
- sub a, b, c → a = b - c
  - ✔ Format:
- Destination, Source1, Source2

 Design Principles
 1. Simplicity supports regularity
✔ Same format for instructions
 2. Make common case fast
✔ Simple instructions used more
 3. Smaller is faster
✔ Registers are faster than memory

 Operands
 Operands are data used in instructions
Types:
✔ Registers
✔ Memory
✔ Constants (Immediate)

 Registers
 Small and fast storage inside CPU
 RISC-V has 32 registers

Examples:

✔ s0–s11 → variables

✔ t0–t6 → temporary

✔ zero → always 0

⍰ Immediate (Constants)

⍰ Values directly inside instruction

Example:

⍰ addi x1, x2, 5

✔ No need to access memory

⍰ Memory

⍰ Large storage but slower

✔ Data must be loaded into registers before use

✔ Byte addressable

✔ 1 word = 4 bytes

 Load and Store (Very Important)
 Load (lw) → Read from memory
 lw x5, 8(x10)
✔ x5 ← Memory[x10 + 8]

 Store (sw) → Write to memory
 sw x5, 8(x10)
✔ Memory[x10 + 8] ← x5

“Assembly language uses simple instructions with registers, memory, and load/store operations to execute programs.”

, Assembler Directives

An **assembler** is a **system software** that **converts assembly language into machine language**.

Assembler directives tell the assembler **how to store data, where to place code, and how to organize a program**.

Assembler directives are **instructions to the assembler**, not the CPU.

They **do not generate machine code** but help in **program organization and data definition**.

Common RISC-V Assembler Directives:

Directive	Purpose
`.data`	Defines data section
`.text`	Defines code (instruction) section
`x.globl label`	Makes a label global
`.word`	Allocates 32-bit data
`.byte`	Allocates 8-bit data
`.half`	Allocates 16-bit data
`.asciz`	Stores null-terminated string
`.align n`	Aligns data in memory
`.end`	Marks end of program

How an Assembler Works

⍰ An assembler converts assembly language → machine code

⍰ **Steps**

⍰ 1. Input

- Programmer writes code in assembly language
- Uses mnemonics like MOV, ADD

⍰ 2. Translation ⭑

- Assembler reads instructions
- Converts mnemonics → binary machine code
- Resolves registers, opcodes, addressing

⍰ 3. Output

- Produces object code (machine code)
- Ready for linking and execution

- **Types of Assembler**
  - Single-pass → one scan (fast)
  - Multi-pass → multiple scans (more accurate)

"Assembler converts assembly instructions into machine code by translating mnemonics and generating object code."

Program Concepts

1) What is Program Concept?

Programs are usually written in **high-level languages** like:

- C
- C++
- Java

These are easy for humans.

Example:

a = b + c;

But CPU cannot understand this directly ✕

CPU only understands **assembly / machine instructions**.

So high-level code is converted into assembly.

Example:

add s0, s1, s2

 This means add values in s1 and s2, store in s0.

⍰ 2) Program Flow (VERY IMPORTANT)

This is the **order in which CPU executes instructions**.

Imagine CPU is reading steps one by one ⍰

⍰ Main idea

- Instructions are stored in memory
- Each instruction = **4 bytes**
- CPU uses **PC (Program Counter)**
- PC stores current instruction address
- After execution:

PC = PC + 4

⍰ Step-by-step flow

1. PC points to instruction
2. CPU fetches instruction
3. CPU decodes meaning
4. CPU executes it
5. PC moves to next instruction
6. Repeat

```
┌──────────┐
│ Start    │
└────┬─────┘
     ↓
┌──────────┐
│ PC points │
│ instruction │
└────┬─────┘
     ↓
┌──────────┐
│ Fetch    │
└────┬─────┘
     ↓
┌──────────┐
│ Decode   │
└────┬─────┘
     ↓
┌──────────┐
│ Execute  │
└────┬─────┘
     ↓
┌──────────┐
│ PC = PC+4 │
└────┬─────┘
     ↓
   Repeat
```

 Example

Address	Instruction
100	addi s1, s2, 5
104	lw s3, 0(s1)

CPU flow:

PC = 100 → execute addi

PC = 104 → execute lw

3) Logical Instructions

Logical instructions are **simple bit operations on 0s and 1s**.

Think of each bit like a **small switch bulb**:

- `1` = ON
- `0` = OFF

These instructions help the CPU **compare, clear, set, or change bits easily**. They are mainly used in **masking, checking flags, and controlling hardware bits**.

In simple words: **logical instructions help the CPU make small binary decisions fast.** These work on **bits (0 and 1)**.

These work on **bits (0 and 1)**.

AND

1010

1100

1000

keeps common 1s only

 OR

1010

1100

1110

 combines all 1s

 XOR

1010

1100

0110

 different bits become 1

✍ **Syntax**

and s1, s2, s3

or s1, s2, s3

xor s1, s2, s3

▯ 4) Shift Instructions

Shift instructions **move the bits inside a register to the left or right ▯**.

shift instructions slide binary bits like moving blocks in a row.

They are very useful for:

- ☒ fast multiplication by 2
- ➗ fast division by 2
- ▯ bit manipulation
- ▯ hardware control

So, shift instructions are a **quick shortcut for math and bit movement inside CPU**.

▯ Left shift
0001 << 2 = 0100
▯ multiply by 4
▯ Right shift
1000 >> 1 = 0100
▯ divide by 2

✍ **Syntax**

1) `sll` → Shift Left Logical

```
sll s1, s2, s3
```

Moves bits **left**

Example:

If `s2 = 0001`
and `s3 = 2`

```
0001 << 2 = 0100
```

Stored in `s1`.

 Left shift = multiply by 2

2) `srl` → Shift Right Logical

```
srl s1, s2, s3
```

Moves bits **right** ➡
fills left empty spaces with `0`.

Example:

```
1000 >> 1 = 0100
```

 Right shift = divide by 2

3) `sra` → Shift Right Arithmetic

```
sra s1, s2, s3
```

Also moves right ➡
but keeps the **sign bit**.

Used for **signed numbers**.

Positive/negative sign stays safe

SRA is a careful right shift that protects the number's sign

Example 1 (Positive Number)

Take:

```
s2 = 0100   (+4)
s3 = 1
```

Now shift right:

```
0100 >> 1 = 0010
```

So:

```
s1 = 0010   (+2)
```

✅ Sign bit `0` stays same

□ Example 2 (Negative Number)

Take:

```
s2 = 1100   (-4)
s3 = 1
```

Now arithmetic right shift:

```
1100 >> 1 = 1110
```

So:

```
s1 = 1110   (-2)
```

✅ Sign bit `1` is copied again on left side
✅ Negative sign remains safe

5) Multiply Instructions

Used for multiplication.

✍ Syntax

mul s3, s1, s2

Example

s1 = 6

s2 = 4

mul s3, s1, s2

Output:

s3 = 24

Important point

- mul → lower 32 bits
- mulh → upper 32 bits

Why mul and mulh?

When two numbers are small, answer fits in one register ✅

Example:

6 × 4 = 24

So:

mul s3, s1, s2

s3 = 24

Easy

Problem with Big Numbers

Now take bigger numbers:

70000 × 70000 = 4900000000

This answer is **very big**

A normal register can store only **32 bits**.
So CPU **cuts the big answer into 2 parts** ✂
Big answer = [top part] [bottom part]

---

mul
mul s3, s1, s2
keeps the **lower / bottom part** only

---

mulh
mulh s4, s1, s2
keeps the **upper / top part**

Super Easy Binary Style Idea
Suppose result is:
123456789ABCDEF
CPU splits like:
Top part = 12345678
Bottom part = 9ABCDEF

- mulh → top
- mul → bottom

One Easy Line
mul stores bottom part, mulh stores top part of big multiplication result.

Division Instruction in RISC-V

The instruction is:

```
div s1, s2, s3
```

 This means:

```
s1 = s2 / s3
```

`s2` = number to divide
`s3` = divisor
`s1` = quotient answer

Easy Example

```
div s1, s2, s3
```

Suppose:

```
s2 = 20
s3 = 4
```

Then:

```
s1 = 5
```

✅ because **20 ÷ 4 = 5**

So division instruction tells CPU

Remainder also important

Sometimes exam asks **remainder too**.

Use:

```
rem s4, s2, s3
```

Example:

```
21 ÷ 4
```

quotient = `5`
remainder = `1`

So:

```
div s1, s2, s3   → 5
rem s4, s2, s3   → 1
```

Branching

Branch instructions **change the normal order of program execution**.
Normally CPU executes instructions one by one in sequence.
But branch instructions change the **Program Counter (PC)** so CPU can:

- skip some instructions
- repeat previous instructions
- jump to another place in the program

So branching helps CPU fetch instructions that are **not in sequential order**.
There are **2 types**:
 1) Conditional Branch
CPU first checks a condition
It jumps **only if the condition is TRUE**.
 2) Unconditional Branch (Jump)
CPU jumps **directly without checking any condition**

Conditional Branch Instructions

RISC-V has **6 conditional branch instructions**.

Each uses:

- 2 source registers
- 1 label (target location)

Types

beq → branch if equal
bne → branch if not equal
blt → branch if less than
bge → branch if greater than or equal
bltu → unsigned less than
bgeu → unsigned greater or equal

- `blt` and `bge` → signed numbers
- `bltu` and `bgeu` → unsigned numbers

Example

```
beq s0, s1, target
addi s2, s2, 1
```

```
sub  s3, s3, s4
target:
add  s5, s5, s6
```

□ What happens here?

If:

```
s0 = s1
```

then `beq` becomes TRUE ✅

So CPU jumps directly to:

```
target:
add s5, s5, s6
```

That means:

```
addi
sub
```

are skipped ❌

Code Example 6.12 CONDITIONAL BRANCHING USING beq

RISC-V Assembly Code

```
 addi  s0, zero, 4       # s0 = 0 + 4 = 4
 addi  s1, zero, 1       # s1 = 0 + 1 = 1
 slli  s1, s1, 2         # s1 = 1 << 2 = 4
 beq   s0, s1, target    # s0 == s1, so branch is taken
 addi  s1, s1, 1         # not executed
 sub   s1, s1, s0        # not executed
target:                  # label
 add   s1, s1, s0        # s1 = 4 + 4 = 8
```

Code Example 6.13 CONDITIONAL BRANCHING USING bne

ktunotes.live
2024 SCHEME NOTES

RISC-V Assembly Code

```
 addi  s0, zero, 4       # s0 = 0 + 4 = 4
 addi  s1, zero, 1       # s1 = 0 + 1 = 1
 slli  s1, s1, 2         # s1 = 1 << 2 = 4
 bne   s0, s1, target    # branch not taken
 addi  s1, s1, 1         # s1 = 4 + 1 = 5
 sub   s1, s1, s0        # s1 = 5 − 4 = 1
target:
 add   s1, s1, s0        # s1 = 1 + 4 = 5
```

the branch is not taken because s0 is equal to s1, and the code continues to execute directly after the bne (branch if not equal) instruction.
All instructions in this code snippet are executed

Jumps (Unconditional Branch)

A **jump** means CPU **changes the Program Counter (PC) directly to another address without checking any**

condition.
 No **if**, no compare, no true/false.
CPU simply says:
**Go directly to target label **
That is why it is called **unconditional branch**.

 Why jump is used
Jump is used when CPU wants to:

- skip some instructions
- go to a function
- continue program from another place
- exit from loop/menu

So instead of going line by line, CPU **takes a shortcut road**

 Main Jump Instructions
RISC-V uses 3 jump instructions:
j
jal
jr

---

 1) j → Direct Jump
This is the simplest jump.
j target
 CPU directly goes to label target
No return address is saved.

▯ Example

```
j target
addi s1, s1, 1
sub  s2, s2, s3
or   s4, s4, s5
target:
add  s6, s6, s7
```

▯ What happens?

CPU executes:

j target

Then it immediately jumps to:

target:

add s6, s6, s7

So these lines are skipped ✕

addi

sub

or

▯ 2) jal → Jump and Link

Used for **function calls** ▯

jal function

▯ jumps to function

▯ also saves return address in ra

So after function finishes, CPU can come back.
Think:
go to function and remember return road

---

3) jr → Jump Register

Here jump address is stored inside a register.
jr ra
 jump to address inside ra
Mostly used to **return from function**.

Code Example 6.14 UNCONDITIONAL BRANCHING USING j

RISC-V Assembly Code

```
 j    target        # jump to target
 srai s1, s1, 2     # not executed
 addi s1, s1, 1     # not executed
 sub  s1, s1, s0    # not executed
target:
 add  s1, s1, s0    # s1 = s1 + s0
```

Conditional Statements

Conditional statements **run code only when a condition is true** ✅
Used in high-level languages:

- if
- if/else
- switch/case

In RISC-V, these are implemented using **branch instructions** ⍰

1) If Statement
An **if statement runs a block only when condition is true**.
⍰ **High-level code**
if(apples == oranges)

fruits = 1;

RISC-V idea

CPU usually checks the **opposite condition to skip block**.

```
bne s1, s2, skip
addi s3, zero, 1
skip:
```

Meaning

- if apples != oranges → skip ❌
- otherwise execute if block ✅

This is exactly what your PDF means by **testing opposite condition**.

Code Example 6.15 IF STATEMENT

High-Level Code	RISC-V Assembly Code

`if (apples == oranges)`
`  f = g + h;`
`apples = oranges - h;`
	`# s0 = apples, s1 = oranges`
	`# s2 = f, s3 = g, s4 = h`
	`  bne s0, s1, L1 # skip if (apples != oranges)`
	`  add s2, s3, s4 # f = g + h`
	`L1: sub s0, s1, s4 # apples = oranges - h`

2) If/Else Statement

Here one of **two blocks** runs.

High-level

```
if(a == b)
  x = 1;
else
  x = 0;
```

RISC-V

```
bne s1, s2, ELSE
addi s3, zero, 1
j END    // After IF block runs, CPU must skip ELSE block.
                                        So it jumps directly to END.
ELSE:
addi s3, zero, 0
END:
```

Meaning

- condition true → first block
- false → else block

```
C code
if (s1 == s2)
   s0 = s0 + 1;
else
   s0 = s0 - 1;
```

```
bne s1, s2, else_part        # if condition false → ELSE
addi s0, s0, 1               # IF part
j end_if                     # skip ELSE

else_part:
addi s0, s0, -1              # ELSE part

end_if:
```

Instruction	Meaning
beq rs1, rs2, label	if rs1 == rs2
bne rs1, rs2, label	if rs1 ≠ rs2
blt rs1, rs2, label	if rs1 < rs2 (signed)
bge rs1, rs2, label	if rs1 ≥ rs2 (signed)
bltu rs1, rs2, label	unsigned <
bgeu rs1, rs2, label	unsigned ≥

3) Switch / Case

Used when there are **many choices**

Example ATM:

- press 1 → ₹20
- press 2 → ₹50
- press 3 → ₹100
- otherwise → default

This is same as **many if/else blocks**.

 Example

```
switch(button){
 case 1: cash=20;
 case 2: cash=50;
 default: cash=100;
}
```

In assembly, CPU uses **multiple compare + branch instructions**.

High-Level Code

```
switch (button) {
  case 1:  amt = 20;  break;

  case 2:  amt = 50;  break;

  case 3:  amt = 100; break;

  default: amt = 0;
}

// equivalent function using
// if/else statements
if      (button == 1)  amt = 20;
else if (button == 2)  amt = 50;
else if (button == 3)  amt = 100;
else                   amt = 0;
```

RISC-V Assembly Code

```
# s0 = button, s1 = amt
case1:
 addi  t0, zero, 1        # t0 = 1
 bne   s0, t0, case2      # button == 1?
 addi  s1, zero, 20       # if yes, amt = 20
 j     done               # break out of case
case2:
 addi  t0, zero, 2        # t0 = 2
 bne   s0, t0, case3      # button == 2?
 addi  s1, zero, 50       # if yes, amt = 50
 j     done               # break out of case
case3:
 addi  t0, zero, 3        # t0 = 3
 bne   s0, t0, default    # button == 3?
 addi  s1, zero, 100      # if yes, amt = 100
 j     done               # break out of case
default:
 add   s1, zero, zero     # amt=0
done:
```

ktunotes.live
2024 S | E | E | S | T | E

4) Loops

A **loop repeats the same block of code again and again** until a condition becomes false.
 Instead of writing same code many times, CPU repeats it automatically.
Main types:

- while
- for
- do/while

Code Example 6.18 WHILE LOOP

High-Level Code

```
// determines the power
// of x such that 2^x = 128
int pow = 1;
int x   = 0;


while (pow != 128) {
  pow = pow * 2;
  x = x + 1;
}
```

RISC-V Assembly Code

```
#  s0 = pow, s1 = x
       addi s0,  zero, 1      # pow = 1
       add  s1,  zero, zero   # x = 0

       addi t0,  zero, 128    # t0 = 128
while: beq  s0,  t0, done     # pow = 128?
       slli s0,  s0, 1        # pow = pow * 2
       addi s1,  s1, 1        # x = x + 1
       j    while             # repeat loop
done:
```

Code Example 6.20 FOR LOOP

High-Level Code

```
// add the numbers from 0 to 9
int sum = 0;
int i;

for (i = 0; i < 10; i = i + 1) {
   sum = sum + i;
}
```

RISC-V Assembly Code

```
# s0 = i, s1 = sum
      addi  s1, zero, 0      # sum = 0
      addi  s0, zero, 0      # i = 0
      addi  t0, zero, 10     # t0 = 10
for:  bge   s0, t0, done     # i >= 10?
      add   s1, s1, s0       # sum = sum + i
      addi  s0, s0, 1        # i = i + 1
      j     for              # repeat loop
done:
```

What is Array?

An **array is a group of similar data stored together**.

Example:
scores = [80, 75, 90, 60]
All are marks, so they are stored as one array

Why array is useful
Instead of using many variables:

score1
score2
score3
score4

we use one name:
scores[ ]
Easy to store and access

How array is stored in memory
Array elements are stored in **sequential memory locations**.
Like boxes in a row

scores[0]
scores[1]
scores[2]
scores[3]

Each box has an **index number**.

Length
Total number of elements.
Example:
[80, 75, 90, 60]
Length = **4**

▯ Base Address

Address of **first element (scores[0])**
This is called **base address**.

Why next address increases by 4
Each integer = **4 bytes**
So next element address becomes:
base + 4
base + 8
base + 12
Because every integer needs 4 bytes.

Memory Box Example
Suppose:
base = 1000
Then:
scores[0] = 1000
scores[1] = 1004
scores[2] = 1008
scores[3] = 1012
▯ every next box +4

Address	Data
174303BC	scores[199]
174303B8	scores[198]
⋮	⋮
174300A4	scores[1]
174300A0	scores[0]

Main Memory

Figure 6.4 Memory holding `scores[200]` starting at base address 0x174300A0

Code Example 6.21 USING A FOR LOOP TO ACCESS AN ARRAY

High-Level Code

```
int i;
int scores[200];

for (i = 0; i < 200; i = i + 1)

  scores[i] = scores[i] + 10;
```

RISC-V Assembly Code

```
# s0 = scores base address, s1 = i

  addi s1, zero, 0    # i = 0
  addi t2, zero, 200  # t2 = 200

for:
  bge  s1, t2, done   # if i >= 200 then done
  slli t0, s1, 2      # t0 = i * 4
  add  t0, t0, s0     # address of scores[i]
  lw   t1, 0(t0)      # t1 = scores[i]
  addi t1, t1, 10     # t1 = scores[i] + 10
  sw   t1, 0(t0)      # scores[i] = t1
  addi s1, s1, 1      # i = i + 1
  j    for            # repeat
done:
```

KTU
ktunotes.live

Function Call?

A function is a **small reusable block of code**.

Instead of writing same code again and again, we make a function.

Example in C:

```
sum(a,b)
```

So if `main()` needs work done, it **calls another function**.

This is called **function call** □

Full Simple Example

```
main:
jal simple
add s1,s2,s3

simple:
jr ra
```

Step-by-step flow

□ **Step 1**

CPU executes:

jal simple

- jump to simple
- save return address in ra

□ **Step 2**

Function runs:

jr ra

- go back to address in ra

Step 3
CPU continues:
add s1,s2,s3
✅after function

Code Example 6.23 FUNCTION CALL WITH ARGUMENTS AND RETURN VALUES

ktunotes.live
2024 SCHEME NOTES

High-Level Code

```
int main(){
  int y;
  . . .



  y = diffofsums(2, 3, 4, 5);
  . . .
}

int diffofsums(int f, int g, int h, int i){
  int result;

  result = (f + g) - (h + i);

  return result;
}
```

RISC-V Assembly Code

```
# s7 = y
main:
   . . .
   addi a0, zero, 2   # argument 0 = 2
   addi a1, zero, 3   # argument 1 = 3
   addi a2, zero, 4   # argument 2 = 4
   addi a3, zero, 5   # argument 3 = 5
   jal  diffofsums    # call function
   add  s7, a0, zero  # y = returned value
   . . .
# s3 = result
diffofsums:
   add  t0, a0, a1    # t0 = f+g
   add  t1, a2, a3    # t1 = h+i
   sub  s3, t0, t1    # result = (f+g)-(h+i)
   add  a0, s3, zero  # put return value in a0
   jr   ra            # return to caller
```

Stack

The **stack** is a special area in memory used as:
 “temporary storage (scratch space) for functions”
It stores things like:

- register values
- local variables
- return addresses (sometimes)

Key Idea: Stack Grows Downwards

- Stack starts at a **high memory address**
- It **grows downward** (towards smaller addresses)

Register used:

sp = stack pointer

sp always points to the **top of the stack**

How stack "expands"

If we want to store 2 words (8 bytes):

sp = sp - 8

Then we store values there.

Example:

Before:

sp = 0xBEFFFAE8

After expansion:

sp = 0xBEFFFAE0

Now we have space for data.

Why do we use the stack?

Because functions can **mess up registers**.

Example problem:

- main uses t0, t1, s3
- function diffofsums also uses them
- → values get overwritten

Code Example 6.24 FUNCTION THAT SAVES REGISTERS ON THE STACK

High-Level Code

```
int diffofsums(int f, int g, int h, int
i){
 int result;

 result = (f + g) - (h + i);

 return result;
}
```

RISC-V Assembly Code

```
# s3 = result
diffofsums:
  addi sp, sp, -12   # make space on stack to
                     # store three registers
  sw   s3, 8(sp)     # save s3 on stack
  sw   t0, 4(sp)     # save t0 on stack
  sw   t1, 0(sp)     # save t1 on stack
  add  t0, a0, a1    # t0 = f + g
  add  t1, a2, a3    # t1 = h + i
  sub  s3, t0, t1    # result = (f + g) - (h + i)
  add  a0, s3, zero  # put return value in a0
  lw   s3, 8(sp)     # restore s3 from stack
  lw   t0, 4(sp)     # restore t0 from stack
  lw   t1, 0(sp)     # restore t1 from stack
  addi sp, sp, 12    # deallocate stack space
  jr   ra            # return to caller
```

ktunotes.live
2024 EDITION NOTES

Preserved vs Non-Preserved Registers (RISC-V)

When a function runs, it uses CPU registers.

But problem is:
 Another function (caller) may already be using them

So RISC-V divides registers into **2 groups** to avoid confusion.

Preserved Registers (IMPORTANT SAFETY ZONE)

Also called: saved registers

Registers:

- s0 – s11
- sp (stack pointer)
- ra (return address)

Meaning

If a function uses these registers, it MUST NOT destroy them.

So it must:

SAVE before using

RESTORE before returning

```
diffofsums:
   sw s3, 0(sp)   # save old value
   ..............................
   lw s3, 0(sp)   # restore old value
```

Non-Preserved Registers (FREE ZONE)

Also called: temporary registers

Registers:

- t0 – t6
- a0 – a7

Meaning:

You can use them freely inside a function

No need to save

No need to restore

```
add t0, a0, a1
add t1, a2, a3
```

Code Example 6.25 FUNCTION THAT SAVES PRESERVED REGISTERS ON THE STACK

RISC-V Assembly Code

```
# s3 = result
diffofsums:
  addi sp, sp, -4     # make space on stack to store one register
  sw   s3, 0(sp)      # save s3 on stack
  add  t0, a0, a1     # t0 = f + g
  add  t1, a2, a3     # t1 = h + i
  sub  s3, t0, t1     # result = (f + g) - (h + i)
  add  a0, s3, zero   # put return value in a0
  lw   s3, 0(sp)      # restore s3 from stack
  addi sp, sp, 4      # deallocate stack space
  jr   ra             # return to caller
```

Table 6.3 **Preserved and nonpreserved registers and memory**

Preserved (*callee*-saved)	Nonpreserved (*caller*-saved)
Saved registers: `s0-s11`	Temporary registers: `t0-t6`
Return address: `ra`	Argument registers: `a0-a7`
Stack pointer: `sp`	
Stack above the stack pointer	Stack below the stack pointer

✔ **If register is:**

 s-register → SAVE & RESTORE

 t or a-register → FREE to use

Leaf vs Nonleaf Function

1) Leaf Function

A **leaf function** is a function that **does NOT call another function**.

Example:

```
diff:
   add t0, a0, a1
   sub a0, t0, a2
   jr ra
```

This function only does its own work

No jal inside it

So it is a **leaf function**

---

2) Nonleaf Function

A **nonleaf function** is a function that **calls another function**

That means inside it, you will see:

jal somefunction

Example

```
main:
   jal diff
```

if diff itself calls another function:

```
diff:
  jal addnums
```

Then diff becomes a **nonleaf function**

▯ Why more complicated?

Because when one function calls another:

- registers may get overwritten ▯
- return address may change ▯
- temporary values may be lost ▯

So we need **save rules**

▯ 1. Caller Save Rule

Before calling another function, the **caller must save temporary registers if it still needs them later**.

These are:

- t0 – t6
- a0 – a7

✅Example

```
sw t0, 0(sp)
jal func
lw t0, 0(sp)
```

▯ save before call

▯ restore after call

▯ 2. Callee Save Rule

The called function (**callee**) must protect:

- s0 – s11
- ra

if it changes them.

✅Example

```
sw s0, 0(sp)
...
lw s0, 0(sp)
jr ra
```

⍰ Recursive Function

A recursive function = function calls itself

Example:

fact(n) = n * fact(n-1)

In assembly it uses:

jal fact

inside fact

So it acts as:

- caller ✅
- callee ✅

That means:

it may need to save **both temporary and saved registers**

Instruction Execution Cycle

The **instruction execution cycle** is the sequence of steps the CPU follows to process **one instruction**.
It repeats continuously while the program runs.
Main stages

1. **Fetch**
2. **Decode**
3. **Execute**
4. **Store / Write Back**

1 Fetch Stage
The CPU **gets the instruction from memory**.
What happens:

- PC = Program Counter holds address of next instruction
- address goes to memory
- instruction is brought into IR (Instruction Register)
- PC moves to next instruction

---

2 Decode Stage
The **Control Unit (CU)** understands what the instruction means.
Example:
ADD R1, R2
CPU understands:
add contents of R1 and R2

---

3 Execute Stage ⚡
The CPU performs the action.
This may be:

- addition
- subtraction
- load
- store
- jump
- compare

Usually **ALU** does this work.
Example:
R1 = R1 + R2

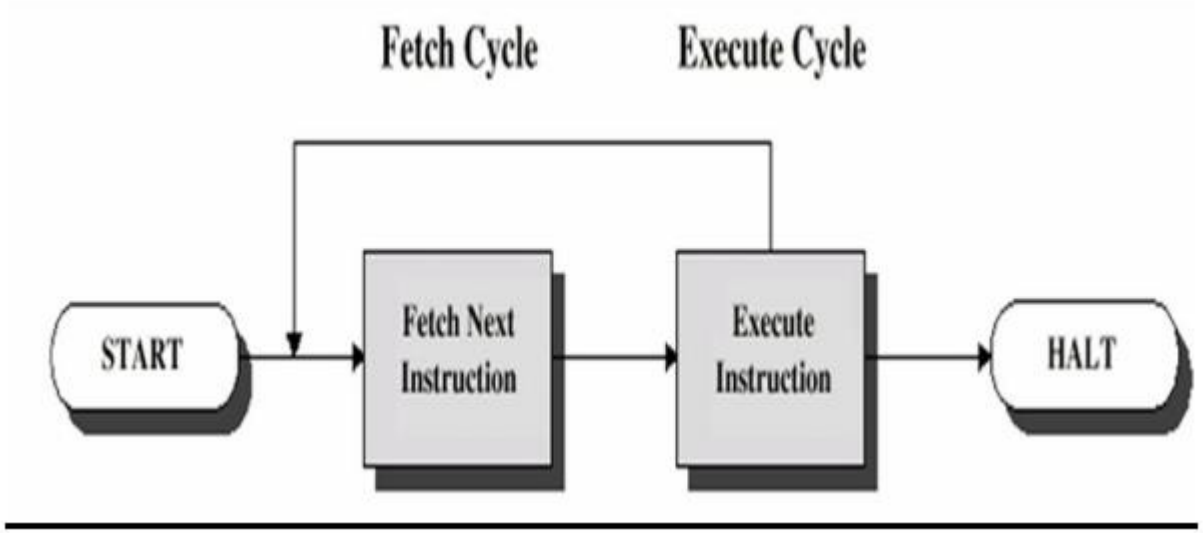
---

4 Store / Write Back
The result is stored back into:

- register
- memory

Example:
R1 = result
Now instruction is complete.

Cycle repeats
After finishing one instruction:
CPU goes back to **Fetch**
This happens **millions or billions of times per second**.



Machine Language

Machine language is the **lowest-level language** that a computer understands directly.
It is made only of:

```
0 and 1
```

These binary digits are called **bits**. The CPU executes them directly.

Different CPUs have different machine language.

Example:

- x86
- ARM
- RISC-V

all use different instruction formats.

Hard for humans Very difficult to:

- read
- write
- debug

Opcode + Operand

Every machine instruction usually has 2 parts:

Opcode → what operation to do
Operand → data/value/address

Example:

```
ADD 45
```

`ADD` = opcode
`45` = operand

RISC-V uses **4 main formats**

1. **R-type**
2. **I-type**
3. **S/B-type**
4. **U/J-type**

R-Type Instructions

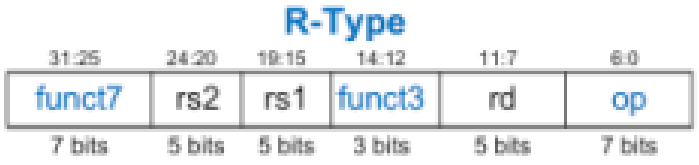
Used for **register-to-register operations**

Examples:

- `add`
- `sub`
- `xor`
- `sll`

These use:

- 2 source registers
- 1 destination register



Field	Meaning
`opcode`	main operation type
`funct3`	sub-operation
`funct7`	exact operation
`rs1`	source register 1
`rs2`	source register 2
`rd`	destination register

Assembly	Field Values						Machine Code						
	funct7	rs2	rs1	funct3	rd	op	funct7	rs2	rs1	funct3	rd	op	
`add s2, s3, s4` `add x18,x19,x20`	0	20	19	0	18	51	0000 000	10100	10011	000	10010	011 0011	(0x01498933)
`sub t0, t1, t2` `sub x5, x6, x7`	32	7	6	0	5	51	0100 000	00111	00110	000	00101	011 0011	(0x407302B3)
	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	

Figure 6.13 Machine code for R-type instructions

Assembly	Field Values						Machine Code						
	funct7	rs2	rs1	funct3	rd	op	funct7	rs2	rs1	funct3	rd	op	
`sll s7, t0, s1` `sll x23,x5, x9`	0	9	5	1	23	51	0000 000	01001	00101	001	10111	011 0011	(0x00929BB3)
`xor s8, s9, s10` `xor x24,x25,x26`	0	26	25	4	24	51	0000 000	11010	11001	100	11000	011 0011	(0x01ACCC33)
	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	

Figure 6.14 More machine code for R-type instructions

R-type instructions have

Assembly	Field Values						Machine Code						
	funct7	rs2	rs1	funct3	rd	op	funct7	rs2	rs1	funct3	rd	op	
`add t3, s4, s5` `add x28,x20,x21`	0	21	20	0	28	51	0000 000	10101	10100	000	11100	011 0011	(0x015A0E33)
	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	

Figure 6.15 Machine code for the R-type instruction of Example 6.3

Instruction	funct7	funct3
add	0000000	000
sub	0100000(32)	000
xor	0000000	100(4)
sll	0000000	001(1)

Add rd, rs1, rs2//format

51 means “R-type ALU operation”

Q Given a 32-bit RISC-V instruction format, show how an assembly-level "Add" operation (e.g., add x5, x6, x7) is translated into machine language.

Ans: Step 1: Identify instruction type

`add` is an **R-type instruction**

So format is:

```
funct7 | rs2 | rs1 | funct3 | rd | opcode
  7       5     5      3       5      7
```

□ Step 2: Fill register fields

Assembly syntax:

```
add rd, rs1, rs2
```

So:

```
rd = x5
rs1 = x6
rs2 = x7
```

Convert to 5-bit binary:

```
x5 = 00101
x6 = 00110
x7 = 00111
```

□ Step 3: Add control bits for `add`

For RISC-V `add`:

```
opcode = 0110011
funct3 = 000
funct7 = 0000000
```

□ Step 4: Place into R-type format

```
funct7   rs2    rs1   funct3   rd    opcode
0000000  00111  00110 000     00101 0110011
```

□ Final 32-bit binary machine code

```
00000000011100110000001010110011
```

□ Hexadecimal form

Group into 4 bits:

```
0000 0000 0111 0011 0000 0010 1011 0011
```

Hex:

```
0x007302B3
```

Assembly:

```
add x5, x6, x7
```

Machine code (binary):

```
00000000011100110000001010110011
```

Machine code (hex):

```
0x007302B3
```

I-Type Instructions (Immediate Type)

I-type means:

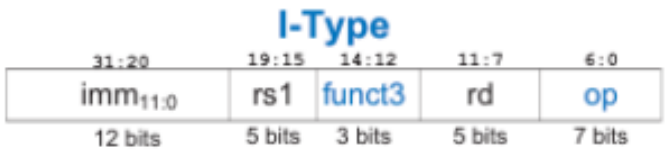
2 register-related fields + 1 immediate value

Used when instruction needs:

- one source register
- one destination register
- one constant number

Common I-type instructions

- `addi`
- `andi`
- `ori`
- `xori`
- `lw`
- `jalr`
- `slli`, `srli`, `srai`



Arithmetic Instructions

Instruction	funct3 (bin)	funct3 (dec)	opcode (bin)	opcode (dec)
`addi`	000	0	0010011	19
`slti`	010	2	0010011	19
`sltiu`	011	3	0010011	19
`xori`	100	4	0010011	19
`ori`	110	6	0010011	19
`andi`	111	7	0010011	19
`slli`	001	1	0010011	19
`srli`	101	5	0010011	19
`srai`	101	5	0010011	19

Load Instructions

Instruction	funct3 (bin)	funct3 (dec)	opcode (bin)	opcode (dec)
`lb`	000	0	0000011	3
`lh`	001	1	0000011	3
`lw`	010	2	0000011	3
`lbu`	100	4	0000011	3
`lhu`	101	5	0000011	3

Jump

Instruction	funct3 (bin)	funct3 (dec)	opcode (bin)	opcode (dec)
`jalr`	000	0	1100111	103

Assembly	Field Values					Machine Code					
	$imm_{11:0}$	rs1	funct3	rd	op	$imm_{11:0}$	rs1	funct3	rd	op	
`addi s0, s1, 12` `addi x8, x9, 12`	12	9	0	8	19	0000 0000 1100	01001	000	01000	001 0011	(0x00C48413)
`addi s2, t1, -14` `addi x18,x6, -14`	-14	6	0	18	19	1111 1111 0010	00110	000	10010	001 0011	(0xFF230913)
`lw t2, -6(s3)` `lw x7, -6(x19)`	-6	19	2	7	3	1111 1111 1010	10011	010	00111	000 0011	(0xFFA9A383)
`lb s4, 0x1F(s4)` `lb x20,0x1F(x20)`	0x1F	20	0	20	3	0000 0001 1111	10100	000	10100	000 0011	(0x01FA0A03)
`slli s2, s7, 5` `slli x18, x23, 5`	5	23	1	18	19	0000 0000 0101	10111	001	10010	001 0011	(0x005B9913)
`srai t1, t2, 29` `srai x6, x7, 29`	(upper 7 bits = 32) 29	7	5	6	19	0100 0001 1101	00111	101	00110	001 0011	(0x41D3D313)
	12 bits	5 bits	3 bits	5 bits	7 bits	12 bits	5 bits	3 bits	5 bits	7 bits	

Figure 6.17 Machine code for I-type instructions

ktunotes.live

Assembly	Field Values					Machine Code					
	$imm_{11:0}$	rs1	funct3	rd	op	$imm_{11:0}$	rs1	funct3	rd	op	
`lw t3, -36(s4)`	-36	20	2	28	3	1111 1101 1100	10100	010	11100	000 0011	(0xFDCA2E03)
`lw x28, -36(x20)`	12 bits	5 bits	3 bits	5 bits	7 bits	12 bits	5 bits	3 bits	5 bits	7 bits	

Figure 6.18 Machine code for the I-type instruction of Example 6.4

S/B-Type Instructions

These instructions use:

- **2 source registers**
- **1 immediate value**

Unlike I-type:
✕no destination register (`rd`)

Because:

- store → writes to memory
- branch → changes PC

31:25	24:20	19:15	14:12	11:7	6:0	
$imm_{11:5}$	rs2	rs1	funct3	$imm_{4:0}$	op	**S-Type**
$imm_{12,10:5}$	rs2	rs1	funct3	$imm_{4:1,11}$	op	**B-Type**
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	

S-Type (Store Instructions)

Used for:

- sb
- sh
- sw

Instruction	funct3 (bin)	funct3 (dec)
`sb`	000	0
`sh`	001	1
`sw`	010	2

Assembly	Field Values						Machine Code						
	$imm_{11:5}$	rs2	rs1	funct3	$imm_{4:0}$	op	$imm_{11:5}$	rs2	rs1	funct3	$imm_{4:0}$	op	
`sw t2, -6(s3)` `sw x7, -6(x19)`	1111 111	7	19	2	11010	35	1111 111	00111	10011	010	11010	010 0011	(0xFE79AD23)
`sh s4, 23(t0)` `sh x20,23(x5)`	0000 000	20	5	1	10111	35	0000 000	10100	00101	001	10111	010 0011	(0x01429BA3)
`sb t5, 0x2D(zero)` `sb x30,0x2D(x0)`	0000 001	30	0	0	01101	35	0000 001	11110	00000	000	01101	010 0011	(0x03E006A3)
	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	

Figure 6.20 Machine code for S-type instructions

B-Type (Branch Instructions)

Used for:

- `beq`
- `bne`
- `blt`
- `bge`
- `bltu`
- `bgeu`

Branch	funct3 (bin)	funct3 (dec)
`beq`	000	0
`bne`	001	1
`blt`	100	4
`bge`	101	5
`bltu`	110	6
`bgeu`	111	7

Figure 6.21 B-type instruction format and calculations for `beq`

U/J-Type Instructions

These use:

- **1 destination register (`rd`)**
- **1 large immediate field (20 bits)**

Used for:

- loading big constants
- long jumps

U-Type Instructions (Upper Immediate)

Used for:

- lui
- auipc

31:12	11:7	6:0	
$imm_{31:12}$	rd	op	**U-Type**
$imm_{20,10:1,11,19:12}$	rd	op	**J-Type**
20 bits	5 bits	7 bits	

U-Type Instructions (Upper Immediate)

Used for:

- lui
- auipc
-

```
lui x5, 0x12345
```

`lui` = **Load Upper Immediate**

 It loads a number into the **upper (top) 20 bits** of the register.

The **lower 12 bits become 0 automatically**.

Think in 32 bits

A register is **32 bits**

```
[ upper 20 bits ][ lower 12 bits ]
```

`lui` fills only the **upper 20 bits**

---

example

```
lui x5, 0x12345
```

The immediate is:

```
0x12345
```

This is **20 bits**

Now CPU shifts it **left by 12 bits**

Result

```
x5 = 0x12345000
```

See the last **3 hex digits = 000**

Because:

```
12 bits = 3 hex digits
```

🔍 Another easy example

```
lui x1, 0xABCDE
```

Result:

```
x1 = 0xABCDE000
```

auipc = **Add Upper Immediate to PC**

It means:

take the **current PC**

- add a **20-bit immediate shifted left by 12**
- store result in register

Example

auipc x5, 0x12345

Suppose current:

PC = 0x10000000

Immediate:

0x12345

Step 1: shift immediate left by 12

Just like lui, add **3 zeros in hex**

0x12345 → 0x12345000

Step 2: add with PC

Now:

x5=0x10000000+0x12345000

So final:

x5 = 0x22345000

⍰ Final answer

auipc x5, 0x12345

If PC = 0x10000000

Example 2: PC = 0x1000
x5=0x1000+0x12345000=

x5 = 0x12346000

J-Type (Jump Type)

Used for **long jumps**

Instruction: jal

```
jal x1, target
```

means:

1. save return address in `x1`
2. jump to `target`

Type	Use	Example
U	load big number	`lui x5, 0x12345`
J	jump far	`jal x1, 16`

Instruction	Type	opcode (bin)	opcode (dec)
`lui`	U	0110111	55
`auipc`	U	0010111	23
`jal`	J	1101111	111

Addressing Modes in RISC-V

Addressing mode specifies the way in which the CPU accesses the operands required for an instruction.
It tells the processor **where the data is located**, whether in a register, inside the instruction, in memory, or relative to the Program Counter (PC).

RISC-V uses different addressing modes to support arithmetic operations, memory access, and control transfer instructions.

1) Register-Only Addressing

In this mode, **all source and destination operands are registers**.

The CPU takes the operands directly from the registers and stores the result in another register.

This mode is used by **R-type instructions**.

Example

add x5, x6, x7

This means:

x5=x6+x7

Here:

- x6 and x7 are source registers
- x5 is the destination register

Features

- very fast
- no memory access required
- commonly used in arithmetic and logical instructions

Examples: add, sub, and, or, xor

2) Immediate Addressing

In this mode, one operand is a **constant value (immediate)** present inside the instruction itself, and the other operand is a register.

This is mainly used by **I-type instructions**.

Example

addi x5, x6, 10

Meaning:

x5=x6+10

Here:

- x6 = source register
- 10 = immediate constant
- x5 = destination register

Features

- avoids loading small constants from memory
- faster execution
- immediate is usually 12-bit signed in RISC-V

Examples: addi, andi, ori, xori, lui

3) Base (Memory) Addressing

This mode is used for **load and store instructions**.

The effective memory address is obtained by adding a **base register** and an **offset immediate**.

Formula

Effective Address=Base Register+Offset

Example

lw x5, 8(x6)

This means:

x5=Memory[x6+8]

Here:

- x6 = base register
- 8 = offset
- x5 = destination register

Features

- used for accessing arrays and variables in memory
- very important for stack operations

Examples: lw, sw, lb, sb

4) PC-Relative Addressing

In this mode, the **branch target address is obtained by adding the signed offset to the current PC value**.

This is used by **branch instructions**.

Formula

New PC=PC+Offset

Example

beq x5, x6, 16

If x5 == x6, then:

PC=PC+16

Features

- useful for loops and conditional statements
- branch destination depends on current PC
- supports position-independent code

Examples: beq, bne, blt, bge

5) Pseudo-Direct Addressing

This addressing mode is used for **jump instructions**.

The instruction provides the jump target as an offset or partially encoded address.

This is mainly used by **J-type instructions**.

Example

jal x1, 100

This means:

- jump to target address
- save return address in x1

Features

- used for function calls and unconditional jumps
- supports long-distance jumps
- return address is automatically stored

Examples: jal, j

Code Example 6.28 CALCULATING THE BRANCH TARGET ADDRESS

MIPS Assembly Code

```
0xA4          beq  $t0, $0, else
0xA8          addi $v0, $0, 1
0xAC          addi $sp, $sp, 8
0xB0          jr   $ra
0xB4    else: addi $a0, $a0, -1
0xB8          jal  factorial
```

Assembly Code	Field Values				Machine Code				
	op	rs	rt	imm	op	rs	rt	imm	
`beq $t0, $0, else`	4	8	0	3	000100	01000	00000	0000 0000 0000 0011	(0x11000003)
	6 bits	5 bits	5 bits	16 bits	6 bits	5 bits	5 bits	16 bits	

Figure 6.28 **Machine code for** `beq`

Example 6.8 CALCULATING THE IMMEDIATE FIELD FOR PC-RELATIVE ADDRESSING

Calculate the immediate field and show the machine code for the branch not equal (`bne`) instruction in the following program.

```
# MIPS assembly code
0x40 loop: add  $t1, $a0, $s0
0x44       lb   $t1, 0($t1)
0x48       add  $t2, $a1, $s0
0x4C       sb   $t1, 0($t2)
0x50       addi $s0, $s0, 1
0x54       bne  $t1, $0, loop
0x58       lw   $s0, 0($sp)
```

Solution: Figure 6.29 shows the machine code for the `bne` instruction. Its branch target address, 0x40, is 6 instructions behind `PC+4` (0x58), so the immediate field is −6.

Assembly Code	Field Values				Machine Code				
	op	rs	rt	imm	op	rs	rt	imm	
`bne $t1, $0, loop`	5	9	0	-6	000101	01001	00000	1111 1111 1111 1010	(0x1520FFFA)
	6 bits	5 bits	5 bits	16 bits	6 bits	5 bits	5 bits	16 bits	

KTU
ktunotes.live
2024 SCHEME N

Evolution of RISC Architecture

RISC (Reduced Instruction Set Computer) was developed to make processors **simpler, faster, and more efficient**.

⍰ Early Computers (Before RISC)

- Used **CISC architecture**
- Had **complex instructions**
- One instruction performed many tasks
- Hardware was **complicated and slow**

⍰ Birth of RISC (1980s)

- Developed by **IBM, Stanford, Berkeley**
- Idea: use **simple instructions** instead of complex ones
- Each instruction performs **only one task**
- Instructions execute in **one clock cycle**

⍰ Key Improvements in RISC

- **Fixed-length instructions** → easy to decode
- **More registers** → faster data access
- **Simple addressing modes**

- Supports **pipelining** for faster execution

⍰ Modern RISC

- Used in **ARM, MIPS, RISC-V**
- Focus on **high speed and low power consumption**
- Widely used in modern devices

Why RISC is used in Embedded Systems

□ Embedded systems need **small, fast, and low-power processors**

□ **Reasons**

• **Low power consumption** □
→ Saves battery (important for mobiles, IoT)

• **Simple design**
→ Less hardware → low cost

• **Fast execution** □
→ Instructions run quickly

• **Efficient pipelining**
→ Improves performance

• **Small size**
→ Fits in small devices

Examples

- Mobile phones
- Embedded controllers
- IoT devices

“RISC evolved from complex CISC systems to provide simple, fast, and energy-efficient processors, making it ideal for modern embedded systems.”

Q Assume a RISC-V based system. Write a short assembly language code snippet to calculate the sum of the first five integers (1 to 5) using a loop and explain the role of the assembler

directives used.

Ans :

```
.data
result: .word 0      # to store final sum

.text
main:
    li x1, 1         # i = 1
    li x2, 5         # limit = 5
    li x3, 0         # sum = 0

loop:
    add x3, x3, x1   # sum = sum + i
    addi x1, x1, 1   # i = i + 1
    ble x1, x2, loop # if i <= 5, repeat

    sw x3, result    # store sum in memory
```

Assembler Directives (Important ⍰)

⍰ **`.data`**

⍰ Defines **data section**
⍰ Used to store variables (result)

⍰ **`.word`**

⍰ Allocates memory for a word (4 bytes)
⍰ Stores initial value (0)

⍰ **`.text`**

⍰ Defines **code section**
⍰ Contains program instructions